NTNU
Kunnskap for en bedre verden

Institutt for datateknologi og informatikk

TDT4145 – Datamodellering og databaser

Prosjektrapport

Forfattere:
Axel Filseth, Leo Egilson Skarbø, Knut Opland Moen

Gruppe 213

3. mars 2026

Innhold

Figurer i

Tabeller i

1 ER-diagram 1

1.1 Antakelser 2

1.2 Det ER-modellen ikke uttrykker 2

2 ER-diagram oversatt til relasjonsskjema 3

2.1 Normalformanalyse 4

3 SQL-script 5

3.1 Kaskadering ved sletting 13

3.2 Restriksjoner håndtert med triggere 13

3.3 Restriksjoner håndtert i applikasjonskode 14

4 Svar på spørsmål 15

4.1 Hvordan (eller kan) dere sørge for at en instruktør ikke kan være til stede på to forskjellige plasser samtidig? Kan dere få til dette ved hjelp av modellen deres, eller må noe slikt implementeres i programvaren 15

4.2 Det samme spørsmålet for en bruker. Kan dette løses i modellen eller må det gjøres i programvaren? 15

4.3 Fra hvilket usecase testes det for / opprettes utestengelse (svartelisting)? Det trenger ikke være en av de oppgitte usecasene. 16

4.4 Statistikk er det noe som må lagres eksplisitt eller går det an å bare gjøre queries for å få svar? Diskuter dette. 16

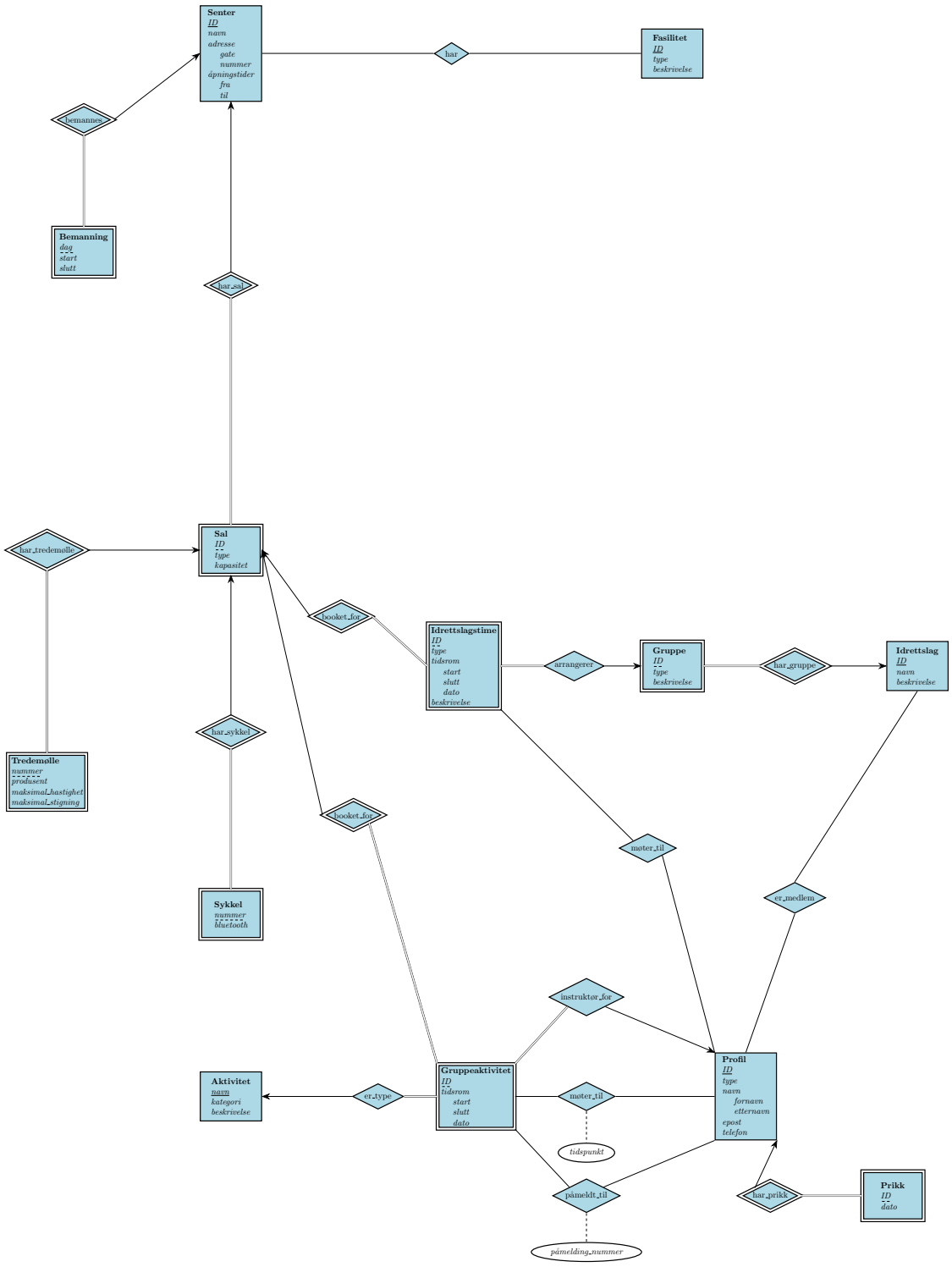
Figurer

1 ER-diagram. 1

Tabeller

1 ER-diagram

ER-diagrammet vist i Figur 1 viser entiteter, relasjoner og attributter.



Figur 1: ER-diagram.

1.1 Antakelser

Følgende antakelser er gjort:

1. **Fasiliteter.** Vi har én entitet for fasiliteter med *type* som attributt (f.eks. badstue, garderobe, øvelsesrom). Problembeskrivelsen tolkes slik at det viktige er å beskrive hvilke fasiliteter senteret har, ikke attributtene til hver fasilitetstype. En alternativ modellering ville vært å splitte opp i flere entity sets. Deltagelsen fra Fasilitet i *har*-relasjonen er partiell, slik at en fasilitet kan eksistere i systemet uten å være tilknyttet et senter. Dette forenkler vedlikehold: fasiliteter kan opprettes på forhånd og fjernes fra et senter uten å måtte slettes fra databasen.

2. **Bemanning.** Bemanningstidspunktene antas like fra uke til uke; *dag* er derfor brukt som diskriminator (eng, discriminator). Vi tolker at det viktige er *når* senteret er bemannet, ikke hvem som bemanner. Bemanning kunne alternativt vært koblet til Profil. Et tredje alternativ ville vært å ha bemanning som en attributt for senteret. Ulempen med dette er at vi fort kunne fått null-verdier.

3. **Åpningstider.** Vi antar at et treningssenter har samme åpningstider hver dag.

4. **Plasser på aktivitet.** Vi antar at antall plasser på en aktivitet er entydig definert ut fra kapasiteten til salen aktiviteten foregår i.

5. **Påmelding og kapasitetsgrense.** Påmelding til en gruppeaktivitet avslås når antall påmeldte har nådd salens kapasitet. Påmeldingsnummeret fungerer som kølapp og brukes til å ordne påmeldte kronologisk. For idrettslagstime brukes et dropin-system der oppmøte avslås når antall oppmøtte har nådd salens kapasitet. Begge kapasitetsgrensene håndheves av triggere i databasen.

6. **Sykler og tredemøller.** Utstyr flyttes ikke mellom rom og identifiseres entydig med nummer og sal.

7. **Instruktør.** Vi har generalisert Profil-entiteten, slik at dette fanger både ansatte/instruktører og vanlige medlemmer. Vi skiller mellom rollene med attributten *type*. Dette kan forsvares med at instruktører potensielt også kan være medlemmer, og dermed delta på andre aktiviteter og være med i idrettslag. I praksis vil typen ansatt funke som et ekstra privilegie der profilen kan være instruktør for en aktivitet, men likevel har alle vanlige medlemsfunksjoner.

8. **Idrettslagstime.** For en undergruppe av et idrettslag som arrangerer idrettslagstime, antas det at en profil ikke trenger å være medlem av gruppen for å delta, altså at det holder at profilen er medlem av idrettslaget.

1.2 Det ER-modellen ikke uttrykker

Følgende regler og restriksjoner kan brukes i et system som bygger på modellen, men uttrykkes ikke eksplisitt i ER-diagrammet:

1. **Prikk ved for sen møting.** En profil får ikke automatisk prikk dersom den møter under 5 minutter før en aktivitet. For å tilrettelegge for prikksystemet skiller vi mellom *påmeldt_til* og *møter_til*, med tidspunkt som attributt i *møter_til*. Da kan man beregne om en profil skal få prikk ved å sjekke om den er påmeldt til en aktivitet uten å møte opp før 5 minutter før start (inkludert ikke møte opp i det hele tatt). Denne regelen er ikke eksplisitt i modellen.

2. **Utestengelse fra nettbooking.** Ved 3 prikker på 30 dager skal profilen utestenges fra nettbooking. Dette kan regnes ut fra en profils prikker og tilhørende datoer. Modellen viser ikke at profilen da ikke kan bruke nettbooking.

3. **Møting etter aktivitetens slutt.** Det skal ikke være mulig å registrere møting til en aktivitet etter aktivitetens slutt. (Vi antar at det er mulig å møte frem til aktivitetens slutt. Det betyr ikke at profilen unngår prikk dersom den møter under 5 minutter før gruppeaktivitetsstart.) Denne restriksjonen vises ikke i ER-modellen.

4. **Maks en booking per sal om gangen.** For et gitt tidspunkt kan en sal bookes av maksimalt én gruppeaktivitet eller én idrettslagstime. Dette er ikke uttrykt i modellen.
5. **Medlemskap for idrettslagstime.** For at en profil skal kunne møte til en idrettslagstime, må den være medlem av idrettslaget. Dette kravet kommer ikke frem i modellen.
6. **Kapasitetsgrense.** Påmelding til en gruppeaktivitet avslås når antall påmeldte er lik salens kapasitet. Tilsvarende avslås oppmøte til en idrettslagstime når antall oppmøtte er lik kapasiteten. Modellen viser ikke disse begrensningene.
7. **48-timersregelen.** En gruppetime skal legges ut 48 timer før den holdes. Denne regelen er en myk forretningsregel som best håndheves i applikasjonskoden, da en hard databaserestriksjon ville hindret instruktører fra å legge ut timer dersom de er noe forsinket, og ikke tillater noen fornuftig margin.

2 ER-diagram oversatt til relasjonsskjema

Relasjonsskjemaet under er utledet direkte fra ER-diagrammet ved standardreglene for ER-til-relasjonsoversetting. Primærnøkler er understreket, fremmednøkler er *kursive*, og attributter som er begge deler er *understreket kursive*.

Fasilitet(ID, type, beskrivelse)

Senter(ID, navn, gate, nummer, fra, til)

Idrettslag(ID, navn, beskrivelse)

Profil(ID, type, fornavn, etternavn, epost, telefon)

Aktivitet(navn, kategori, beskrivelse)

har_fasilitet(*senter_ID*, *fasilitet_ID*)

- *senter_ID* → Senter(ID)
- *fasilitet_ID* → Fasilitet(ID)

Bemanning(*senter_ID*, dag, start, slutt)

- *senter_ID* → Senter(ID)

Sal(*senter_ID*, ID, type, kapasitet)

- *senter_ID* → Senter(ID)

Tredemølle(*senter_ID*, *sal_ID*, nummer, produsent, maks_hastighet, maks_stigning)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)

Sykkel(*senter_ID*, *sal_ID*, nummer, bluetooth)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)

Gruppe(*idrettslags_ID*, ID, type, beskrivelse)

- *idrettslags_ID* → Idrettslag(ID)

Prikk(*profil_ID*, ID, dato)

- *profil_ID* → Profil(ID)

er_medlem(*idrettslags_ID*, *profil_ID*)

- *idrettslags_ID* → Idrettslag(ID)
- *profil_ID* → Profil(ID)

Idrettslagstime(*senter_ID*, *sal_ID*, ID, type, start, slutt, dato, beskrivelse, *idrettslags_ID*, *gruppe_ID*)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)
- *idrettslags_ID* → Idrettslag(ID)
- (*idrettslags_ID*, *gruppe_ID*) → Gruppe(idrettslags_ID, ID)

møter_til_idrett(*senter_ID*, *sal_ID*, *idrettslagstime_ID*, *profil_ID*)

- (*senter_ID*, *sal_ID*, *idrettslagstime_ID*) → Idrettslagstime(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

Gruppeaktivitet(*senter_ID*, *sal_ID*, ID, start, slutt, dato, *aktivitet_navn*, *instrukt_ID*)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)
- *aktivitet_navn* → Aktivitet(navn)
- *instrukt_ID* → Profil(ID)

møter_til_gruppe(*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*, *profil_ID*, tidspunkt)

- (*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*) → Gruppeaktivitet(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

påmeldt_til(*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*, *profil_ID*, påmelding_nummer)

- (*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*) → Gruppeaktivitet(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

2.1 Normalformanalyse

Alle tabellene er på **BCNF** (Boyce–Codd normalform). En relasjon er på BCNF dersom venstre side i enhver ikke-triviell funksjonell avhengighet er en supernøkkel. Nedenfor begrunnes dette gruppevis.

Sterke entiteter – Fasilitet, Idrettslag, Aktivitet. Disse har én enkeltattributt som primærnøkkel (*ID*, resp. *navn* for Aktivitet). Alle øvrige attributter bestemmes entydig av primærnøkkelen, og det finnes ingen andre funksjonelle avhengigheter. Dermed er venstre side i enhver ikke-triviell FD en supernøkkel, og tabellene er på BCNF.

Senter. Primærnøkkelen er *ID*. I tillegg har *navn* en `UNIQUE`-restriksjon, slik at *navn* → *ID, gate, nummer, fra, til* også er en ikke-triviell FD. Venstresiden er en kandidatnøkkel og dermed en supernøkkel, så BCNF er oppfylt. For *(gate, nummer)* kunne man vurdert kandidatnøkkelstatus, men vi antar at dette ikke holder: et senter kan i prinsippet flytte eller to sentre kan midlertidig befinne seg i samme bygg. Adressen er dermed ikke garantert unik og regnes ikke som kandidatnøkkel.

Profil. Primærnøkkelen er *ID*. I tillegg er både *epost* og *telefon* unike (`UNIQUE`) for hver profil, slik at *epost* → *ID, type, fornavn, etternavn, telefon* og *telefon* → *ID, type, fornavn, etternavn, epost* begge er ikke-trivielle FD-er. Venstresidene er kandidatnøkler og dermed supernøkler, så BCNF er oppfylt.

Svake entiteter – Bemanning, Sal, Tredemølle, Sykkel, Idrettslagstime, Gruppeaktivitet, Gruppe, Prikk. ER-til-relasjonsoversettingen av svake entiteter gir en sammensatt primærnøkkel bestående av eierens primærnøkkel og diskriminatoren. De eneste ikke-trivielle funksjonelle avhengighetene er av formen primærnøkkel → øvrige attributter, der venstresiden er primærnøkkelen og dermed en supernøkkel. BCNF er derfor oppfylt for alle disse tabellene.

M:N-relasjoner uten ekstra attributter – har_fasilitet, er_medlem, møter_til_idrett. Alle attributter inngår i primærnøkkelen. Det finnes ingen ikke-trivielle funksjonelle avhengigheter utover de trivielle, og tabellene er derfor trivielt på BCNF.

møter_til_gruppe. Primærnøkkelen er sammensatt av alle fire fremmednøkkelattributtene. *tidspunkt* avhenger funksjonelt av hele primærnøkkelen, og det finnes ingen andre ikke-trivielle FD-er. Tabellen er på BCNF.

påmeldt_til. Primærnøkkelen er (*senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID*). *påmelding_nummer* er unikt innenfor én gruppeaktivitet (kølapp-semantikk), slik at (*senter_ID, sal_ID, gruppeaktivitet_ID, påmelding_nummer*) → *profil_ID* også er en ikke-triviell FD. Venstresiden er en kandidatnøkkel og dermed en supernøkkel. Begge ikke-trivielle FD-er har altså en supernøkkel på venstresiden, og tabellen er på BCNF.

3 SQL-script

SQL-scriptet under oppretter alle tabellene i databasen med primær- og fremmednøkler, `CHECK`-/`UNIQUE`-restriksjoner samt triggere. Scriptet er skrevet for SQLite.

Listing 1: SQL-script for opprettelse av tabeller.

```
-- TDT4145 Prosjektoppgave - SQL-script (oppgave c)
-- Oppretter alle tabeller med primær-/fremmednøkler og restriksjoner.
--
-- Målplattform: SQLite  (PRAGMA foreign_keys = ON må kjøres per tilkobling)
--
-- ============================================================================
-- Restriksjoner som IKKE kan uttrykkes i SQL og må håndteres i applikasjonskode:
--
--   1. Prikk ved manglende/for sen oppmøte: En profil som er påmeldt en
--      gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start
--      skal få en prikk. Regelen krever tidsberegning utenfor databasen.
--
--   2. 48-timersregelen: En gruppetime skal legges ut 48 timer før den
--      holdes. Regelen er en myk forretningsregel som best håndheves i
--      applikasjonskoden, da en hard constraint ville hindret instruktører
--      som er noe forsinket fra å legge ut timer.
--
-- Restriksjoner håndhevet med triggere (se nederst i filen):
--
--   2. Utestengelse (svartelisting): >= 3 prikker siste 30 dager blokkerer
--      ny påmelding (check_utestengelse_insert).
--   3. Oppmøte etter aktivitetens slutt
--   4. Én booking per sal per tidspunkt (4 triggere: insert/update x 2 tabeller)
--   5. Medlemskap for idrettslagstime
--   6. Avbestillingsfrist (senest 1 time før start)
--   7. Instruktørrolle (kun ansatte kan settes som instruktør)
--   8. En instruktør kan ikke lede to gruppeaktiviteter som overlapper i tid
```

```sql
--      (check_instruktør_overlapp_insert/update)
--    9. En bruker kan ikke være påmeldt to gruppeaktiviteter som overlapper i tid
--      (check_bruker_overlapp_påmeldt_insert/update)
--   10. Kapasitetsgrense: Påmelding til gruppeaktivitet avslås når antall
--      påmeldte = salens kapasitet (check_kapasitet_påmeldt_insert)
--   11. Kapasitetsgrense dropin: Oppmøte til idrettslagstime avslås når
--      antall oppmøtte = salens kapasitet (check_kapasitet_idrett_insert)
--
-- ============================================================================

PRAGMA foreign_keys = ON;

-- ============================================================================
-- Sterke entiteter
-- ============================================================================

CREATE TABLE Fasilitet (
    ID          INTEGER PRIMARY KEY AUTOINCREMENT,
    type        TEXT    NOT NULL,
    beskrivelse TEXT
);

CREATE TABLE Senter (
    ID     INTEGER PRIMARY KEY AUTOINCREMENT,
    navn   TEXT    NOT NULL UNIQUE,
    gate   TEXT    NOT NULL,
    nummer TEXT    NOT NULL,
    fra    TIME    NOT NULL,
    til    TIME    NOT NULL,
    CHECK (fra < til)
);

CREATE TABLE Idrettslag (
    ID          INTEGER PRIMARY KEY AUTOINCREMENT,
    navn        TEXT    NOT NULL,
    beskrivelse TEXT
);

CREATE TABLE Profil (
    ID       INTEGER PRIMARY KEY AUTOINCREMENT,
    type     TEXT    NOT NULL CHECK (type IN ('medlem', 'ansatt')),
    fornavn  TEXT    NOT NULL,
    etternavn TEXT   NOT NULL,
    epost    TEXT    NOT NULL UNIQUE,
    telefon  TEXT    NOT NULL UNIQUE
);

CREATE TABLE Aktivitet (
    navn        TEXT PRIMARY KEY,
    kategori    TEXT NOT NULL,
    beskrivelse TEXT
);

-- ============================================================================
-- Svake entiteter og M:N-relasjoner
-- ============================================================================

CREATE TABLE har_fasilitet (
    senter_ID    INTEGER NOT NULL,
    fasilitet_ID INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, fasilitet_ID),
    FOREIGN KEY (senter_ID)    REFERENCES Senter(ID)    ON DELETE CASCADE,
    FOREIGN KEY (fasilitet_ID) REFERENCES Fasilitet(ID) ON DELETE CASCADE
);

CREATE TABLE Bemanning (
    senter_ID INTEGER NOT NULL,
    dag       TEXT    NOT NULL CHECK (dag IN (
                  'mandag','tirsdag','onsdag','torsdag',
                  'fredag','lørdag','søndag')),
    start     TIME    NOT NULL,
    slutt     TIME    NOT NULL,
    PRIMARY KEY (senter_ID, dag),
```

```sql
    FOREIGN KEY (senter_ID) REFERENCES Senter(ID) ON DELETE CASCADE,
    CHECK (start < slutt)
);

CREATE TABLE Sal (
    senter_ID INTEGER NOT NULL,
    ID        INTEGER NOT NULL,
    type      TEXT    NOT NULL,
    kapasitet INTEGER NOT NULL CHECK (kapasitet > 0),
    PRIMARY KEY (senter_ID, ID),
    FOREIGN KEY (senter_ID) REFERENCES Senter(ID) ON DELETE CASCADE
);

CREATE TABLE Tredemølle (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    nummer         INTEGER NOT NULL,
    produsent      TEXT    NOT NULL,
    maks_hastighet REAL    NOT NULL CHECK (maks_hastighet > 0),
    maks_stigning  REAL    NOT NULL CHECK (maks_stigning  > 0),
    PRIMARY KEY (senter_ID, sal_ID, nummer),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID) ON DELETE CASCADE
);

CREATE TABLE Sykkel (
    senter_ID INTEGER NOT NULL,
    sal_ID    INTEGER NOT NULL,
    nummer    INTEGER NOT NULL,
    bluetooth INTEGER NOT NULL DEFAULT 0 CHECK (bluetooth IN (0, 1)),
    PRIMARY KEY (senter_ID, sal_ID, nummer),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID) ON DELETE CASCADE
);

CREATE TABLE Gruppe (
    idrettslags_ID INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    type           TEXT    NOT NULL,
    beskrivelse    TEXT,
    PRIMARY KEY (idrettslags_ID, ID),
    FOREIGN KEY (idrettslags_ID) REFERENCES Idrettslag(ID) ON DELETE CASCADE
);

CREATE TABLE Prikk (
    profil_ID INTEGER NOT NULL,
    ID        INTEGER NOT NULL,
    dato      DATE    NOT NULL,
    PRIMARY KEY (profil_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID) ON DELETE CASCADE
);

CREATE TABLE er_medlem (
    idrettslags_ID INTEGER NOT NULL,
    profil_ID      INTEGER NOT NULL,
    PRIMARY KEY (idrettslags_ID, profil_ID),
    FOREIGN KEY (idrettslags_ID) REFERENCES Idrettslag(ID) ON DELETE CASCADE,
    FOREIGN KEY (profil_ID)      REFERENCES Profil(ID)     ON DELETE CASCADE
);

CREATE TABLE Idrettslagstime (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    type           TEXT    NOT NULL,
    start          TIME    NOT NULL,
    slutt          TIME    NOT NULL,
    dato           DATE    NOT NULL,
    beskrivelse    TEXT,
    idrettslags_ID INTEGER NOT NULL,
    gruppe_ID      INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, ID),
    FOREIGN KEY (senter_ID, sal_ID)            REFERENCES Sal(senter_ID, ID)
                    ON DELETE CASCADE,
    FOREIGN KEY (idrettslags_ID)               REFERENCES Idrettslag(ID)
```

```sql
                        ON DELETE CASCADE,
    FOREIGN KEY (idrettslags_ID, gruppe_ID)    REFERENCES Gruppe(idrettslags_ID, ID)    ON DELETE CASCADE,
    CHECK (start < slutt)
);

CREATE TABLE møter_til_idrett (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    idrettslagstime_ID INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, idrettslagstime_ID, profil_ID),
    FOREIGN KEY (senter_ID, sal_ID, idrettslagstime_ID)
        REFERENCES Idrettslagstime(senter_ID, sal_ID, ID) ON DELETE CASCADE,
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)         ON DELETE CASCADE
);

CREATE TABLE Gruppeaktivitet (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    start          TIME    NOT NULL,
    slutt          TIME    NOT NULL,
    dato           DATE    NOT NULL,
    aktivitet_navn TEXT    NOT NULL,
    instrukt_ID    INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, ID),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID) ON DELETE CASCADE,
    FOREIGN KEY (aktivitet_navn)    REFERENCES Aktivitet(navn),
    FOREIGN KEY (instrukt_ID)       REFERENCES Profil(ID),
    CHECK (start < slutt)
);

CREATE TABLE møter_til_gruppe (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    gruppeaktivitet_ID INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    tidspunkt           DATETIME NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID),
    FOREIGN KEY (senter_ID, sal_ID, gruppeaktivitet_ID)
        REFERENCES Gruppeaktivitet(senter_ID, sal_ID, ID) ON DELETE CASCADE,
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)         ON DELETE CASCADE
);

CREATE TABLE påmeldt_til (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    gruppeaktivitet_ID INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    påmelding_nummer    INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID),
    UNIQUE (senter_ID, sal_ID, gruppeaktivitet_ID, påmelding_nummer),
    FOREIGN KEY (senter_ID, sal_ID, gruppeaktivitet_ID)
        REFERENCES Gruppeaktivitet(senter_ID, sal_ID, ID) ON DELETE CASCADE,
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)         ON DELETE CASCADE
);

-- ============================================================================
-- Triggere
-- ============================================================================

CREATE TRIGGER check_møter_til_gruppe_etter_slutt
BEFORE INSERT ON møter_til_gruppe
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Kan ikke registrere oppmøte etter gruppeaktivitetens slutt.')
    WHERE NEW.tidspunkt > (
        SELECT dato || ' ' || slutt
        FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
```

```sql
          AND sal_ID     = NEW.sal_ID
          AND ID         = NEW.gruppeaktivitet_ID
    );
END;

CREATE TRIGGER check_møter_til_idrett_etter_slutt
BEFORE INSERT ON møter_til_idrett
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Kan ikke registrere oppmøte etter idrettslagstime er ferdig.')
    WHERE datetime('now', 'localtime') > (
        SELECT dato || ' ' || slutt
        FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND ID        = NEW.idrettslagstime_ID
    );
END;

CREATE TRIGGER check_sal_booking_gruppe_insert
BEFORE INSERT ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Salen er allerede booket av en gruppeaktivitet i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
    SELECT RAISE(ABORT, 'Salen er allerede booket av en idrettslagstime i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
END;

CREATE TRIGGER check_sal_booking_idrett_insert
BEFORE INSERT ON Idrettslagstime
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Salen er allerede booket av en gruppeaktivitet i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
    SELECT RAISE(ABORT, 'Salen er allerede booket av en annen idrettslagstime i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
END;

CREATE TRIGGER check_sal_booking_gruppe_update
```

```sql
BEFORE UPDATE ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Salen er allerede booket av en annen gruppeaktivitet i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NOT (senter_ID = NEW.senter_ID AND sal_ID = NEW.sal_ID AND ID = NEW.ID)
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
    SELECT RAISE(ABORT, 'Salen er allerede booket av en idrettslagstime i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
END;

CREATE TRIGGER check_sal_booking_idrett_update
BEFORE UPDATE ON Idrettslagstime
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Salen er allerede booket av en gruppeaktivitet i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
    SELECT RAISE(ABORT, 'Salen er allerede booket av en annen idrettslagstime i dette tidsrommet.')
    WHERE EXISTS (
        SELECT 1 FROM Idrettslagstime
        WHERE senter_ID = NEW.senter_ID
          AND sal_ID    = NEW.sal_ID
          AND dato      = NEW.dato
          AND NOT (senter_ID = NEW.senter_ID AND sal_ID = NEW.sal_ID AND ID = NEW.ID)
          AND NEW.start < slutt
          AND NEW.slutt > start
    );
END;

CREATE TRIGGER check_medlemskap_idrett_insert
BEFORE INSERT ON møter_til_idrett
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profil må være medlem av idrettslaget for å møte til idrettslagstime.')
    WHERE NOT EXISTS (
        SELECT 1 FROM er_medlem em
        JOIN Idrettslagstime it ON it.idrettslags_ID = em.idrettslags_ID
        WHERE it.senter_ID = NEW.senter_ID
          AND it.sal_ID    = NEW.sal_ID
          AND it.ID        = NEW.idrettslagstime_ID
          AND em.profil_ID = NEW.profil_ID
    );
END;

CREATE TRIGGER check_medlemskap_idrett_update
BEFORE UPDATE ON møter_til_idrett
```

```sql
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profil må være medlem av idrettslaget for å møte til idrettslagstime.')
    WHERE NOT EXISTS (
        SELECT 1 FROM er_medlem em
        JOIN Idrettslagstime it ON it.idrettslags_ID = em.idrettslags_ID
        WHERE it.senter_ID = NEW.senter_ID
          AND it.sal_ID    = NEW.sal_ID
          AND it.ID        = NEW.idrettslagstime_ID
          AND em.profil_ID = NEW.profil_ID
    );
END;

CREATE TRIGGER check_avbestillingsfrist
BEFORE DELETE ON påmeldt_til
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Avbestilling må skje senest 1 time før aktivitetens start.')
    WHERE datetime('now', 'localtime') > (
        SELECT datetime(dato || ' ' || start, '-1 hour')
        FROM Gruppeaktivitet
        WHERE senter_ID = OLD.senter_ID
          AND sal_ID    = OLD.sal_ID
          AND ID        = OLD.gruppeaktivitet_ID
    );
END;

CREATE TRIGGER check_instruktør_rolle_insert
BEFORE INSERT ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Instruktør må ha type ansatt.')
    WHERE (SELECT type FROM Profil WHERE ID = NEW.instrukt_ID) != 'ansatt';
END;

CREATE TRIGGER check_instruktør_rolle_update
BEFORE UPDATE ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Instruktør må ha type ansatt.')
    WHERE (SELECT type FROM Profil WHERE ID = NEW.instrukt_ID) != 'ansatt';
END;

CREATE TRIGGER check_instruktør_overlapp_insert
BEFORE INSERT ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Instruktøren leder allerede en annen gruppeaktivitet som overlapper i tid.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE instrukt_ID = NEW.instrukt_ID
          AND dato        = NEW.dato
          AND NEW.start   < slutt
          AND NEW.slutt   > start
    );
END;

CREATE TRIGGER check_instruktør_overlapp_update
BEFORE UPDATE ON Gruppeaktivitet
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Instruktøren leder allerede en annen gruppeaktivitet som overlapper i tid.')
    WHERE EXISTS (
        SELECT 1 FROM Gruppeaktivitet
        WHERE instrukt_ID = NEW.instrukt_ID
          AND dato        = NEW.dato
          AND NOT (senter_ID = NEW.senter_ID AND sal_ID = NEW.sal_ID AND ID = NEW.ID)
          AND NEW.start   < slutt
```

```sql
          AND NEW.slutt   > start
    );
END;


CREATE TRIGGER check_bruker_overlapp_påmeldt_insert
BEFORE INSERT ON påmeldt_til
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profilen er allerede påmeldt en annen gruppeaktivitet som overlapper i tid.')
    WHERE EXISTS (
        SELECT 1
        FROM påmeldt_til pt
        JOIN Gruppeaktivitet ny  ON  ny.senter_ID = NEW.senter_ID
                                 AND ny.sal_ID    = NEW.sal_ID
                                 AND ny.ID        = NEW.gruppeaktivitet_ID
        JOIN Gruppeaktivitet eks ON eks.senter_ID = pt.senter_ID
                                 AND eks.sal_ID   = pt.sal_ID
                                 AND eks.ID       = pt.gruppeaktivitet_ID
        WHERE pt.profil_ID = NEW.profil_ID
          AND NOT (pt.senter_ID          = NEW.senter_ID
               AND pt.sal_ID             = NEW.sal_ID
               AND pt.gruppeaktivitet_ID = NEW.gruppeaktivitet_ID)
          AND eks.dato = ny.dato
          AND ny.start < eks.slutt
          AND ny.slutt > eks.start
    );
END;

CREATE TRIGGER check_bruker_overlapp_påmeldt_update
BEFORE UPDATE ON påmeldt_til
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profilen er allerede påmeldt en annen gruppeaktivitet som overlapper i tid.')
    WHERE EXISTS (
        SELECT 1
        FROM påmeldt_til pt
        JOIN Gruppeaktivitet ny  ON  ny.senter_ID = NEW.senter_ID
                                 AND ny.sal_ID    = NEW.sal_ID
                                 AND ny.ID        = NEW.gruppeaktivitet_ID
        JOIN Gruppeaktivitet eks ON eks.senter_ID = pt.senter_ID
                                 AND eks.sal_ID   = pt.sal_ID
                                 AND eks.ID       = pt.gruppeaktivitet_ID
        WHERE pt.profil_ID = NEW.profil_ID
          AND NOT (pt.senter_ID          = NEW.senter_ID
               AND pt.sal_ID             = NEW.sal_ID
               AND pt.gruppeaktivitet_ID = NEW.gruppeaktivitet_ID)
          AND eks.dato = ny.dato
          AND ny.start < eks.slutt
          AND ny.slutt > eks.start
    );
END;

CREATE TRIGGER check_utestengelse_insert
BEFORE INSERT ON påmeldt_til
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profilen er utestengt fra nettbooking: 3 eller flere prikker de siste 30 dagene.')
    WHERE (
        SELECT COUNT(*) FROM Prikk
        WHERE profil_ID = NEW.profil_ID
          AND dato >= date('now', 'localtime', '-30 days')
    ) >= 3;
END;

CREATE TRIGGER check_utestengelse_update
BEFORE UPDATE ON påmeldt_til
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Profilen er utestengt fra nettbooking: 3 eller flere
```

```
        prikker de siste 30 dagene.')
    WHERE (
        SELECT COUNT(*) FROM Prikk
        WHERE profil_ID = NEW.profil_ID
          AND dato >= date('now', 'localtime', '-30 days')
    ) >= 3;
END;

CREATE TRIGGER check_kapasitet_påmeldt_insert
BEFORE INSERT ON påmeldt_til
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Gruppeaktiviteten er full - kapasitetsgrensen er nådd.')
    WHERE (
        SELECT COUNT(*) FROM påmeldt_til
        WHERE senter_ID          = NEW.senter_ID
          AND sal_ID             = NEW.sal_ID
          AND gruppeaktivitet_ID = NEW.gruppeaktivitet_ID
    ) >= (
        SELECT kapasitet FROM Sal
        WHERE senter_ID = NEW.senter_ID
          AND ID        = NEW.sal_ID
    );
END;

CREATE TRIGGER check_kapasitet_idrett_insert
BEFORE INSERT ON møter_til_idrett
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Idrettslagstimen er full - kapasitetsgrensen er nådd.')
    WHERE (
        SELECT COUNT(*) FROM møter_til_idrett
        WHERE senter_ID          = NEW.senter_ID
          AND sal_ID             = NEW.sal_ID
          AND idrettslagstime_ID = NEW.idrettslagstime_ID
    ) >= (
        SELECT kapasitet FROM Sal
        WHERE senter_ID = NEW.senter_ID
          AND ID        = NEW.sal_ID
    );
END;
```

3.1 Kaskadering ved sletting

De fleste fremmednøkler er definert med `ON DELETE CASCADE`. Svake entiteter (f.eks. *Sal*, *Bemanning*, *Prikk*, *Tredemølle*) eksisterer kun i kraft av sin eiertabell, og bør fjernes dersom eieren slettes. Det samme gjelder koblings- og oppmøtetabeller (*har_fasilitet*, *er_medlem*, *påmeldt_til*, *møter_til_gruppe* osv.): en rad i en slik tabell er meningsløs uten begge sider av relasjonen.

To fremmednøkler beholder standard oppførsel (`NO ACTION`, dvs. sletting avvises):

- *Gruppeaktivitet.aktivitet_navn* → *Aktivitet*: en aktivitetstype er en gjenbrukbar definisjon og ikke en eier. Sletting av f.eks. «Yoga» skal ikke stille fjerne alle planlagte yogatimer.
- *Gruppeaktivitet.instrukt_ID* → *Profil*: sletting av en instruktørprofil skal avvises så lenge det finnes gruppeaktiviteter knyttet til profilen, slik at aktivitetene må avlyses eller tildeles en annen instruktør først.

3.2 Restriksjoner håndtert med triggere

Følgende restriksjoner krever subspørringer og kan ikke uttrykkes med `CHECK`-restriksjoner i SQLite (SQLite støtter ikke subspørringer i `CHECK`). De håndheves i stedet med `BEFORE INSERT/UPDATE/DELETE`-triggere:

1. **Utestengelse (svartelisting).** Dersom en profil har ≥ 3 prikker innenfor de siste 30 dagene, utestenges den fra nettbooking. Triggeren `check_utestengelse_insert` gjør at profiler med for mange prikker ikke kan melde seg på en aktivitet.

2. **Oppmøte etter aktivitetens slutt.** Det skal ikke være mulig å registrere oppmøte til en aktivitet etter at den er ferdig. Triggerne `check_møter_til_gruppe_etter_slutt` og `check_møter_til_idrett_etter_slutt` avviser slike innsettinger.

3. **Én booking per sal per tidspunkt.** For et gitt tidsrom kan en sal kun ha én gruppeaktivitet eller én idrettslagstime. Fire triggere (insert/update for hvert av de to tabellene) sjekker at aktiviteter ikke overlapper med eksisterende bookinger.

4. **Medlemskap for idrettslagstime.** En profil må være medlem av idrettslaget (via `er_medlem`) for å møte til en idrettslagstime.

5. **Avbestillingsfrist.** Avbestilling av en gruppeaktivitet må skje senest 1 time før aktivitetens start. Triggeren `check_avbestillingsfrist` avviser slettinger fra `påmeldt_til` dersom fristen er utløpt.

6. **Instruktørrolle.** Kun profiler med `type = 'ansatt'` kan settes som instruktør for en gruppeaktivitet.

7. **Instruktøroverlapp.** En instruktør kan ikke lede to gruppeaktiviteter som overlapper i tid. Triggerne `check_instruktør_overlapp_insert` og `check_instruktør_overlapp_update` sjekker om det finnes en annen gruppeaktivitet med samme instruktør og overlappende tidsrom.

8. **Brukeroverlapp ved påmelding.** En profil kan ikke være påmeldt to gruppeaktiviteter som overlapper i tid. Triggerne `check_bruker_overlapp_påmeldt_insert` og `check_bruker_overlapp_påmeldt_up` gjør en `JOIN` mot `Gruppeaktivitet` for å sammenligne tidsrom og avviser påmeldingen dersom det finnes en kollisjon.

9. **Kapasitetsgrense for gruppeaktivitet.** Påmelding til en gruppeaktivitet avslås når antall påmeldte er lik salens kapasitet. Triggeren `check_kapasitet_påmeldt_insert` teller eksisterende rader i `påmeldt_til` og sammenligner med `Sal.kapasitet`.

10. **Kapasitetsgrense for idrettslagstime.** Oppmøte til en idrettslagstime (dropin) avslås når antall oppmøtte er lik salens kapasitet. Triggeren `check_kapasitet_idrett_insert` teller eksisterende rader i `møter_til_idrett` og sammenligner med `Sal.kapasitet`.

3.3 Restriksjoner håndtert i applikasjonskode

Følgende restriksjoner kan ikke håndheves i databasen og må håndteres av applikasjonsprogrammene:

1. **Prikk ved manglende/for sen oppmøte.** En profil som er påmeldt en gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start, skal få en prikk. Regelen krever tidsberegning og ekstern logikk for å utløse prikken. En mulig løsning er å en gang i døgnet gjøre en sjekk på alle påmeldte profiler for en aktivitet, og gi prikk til de som ikke har møtt opp.

4 Svar på spørsmål

4.1 Hvordan (eller kan) dere sørge for at en instruktør ikke kan være til stede på to forskjellige plasser samtidig? Kan dere få til dette ved hjelp av modellen deres, eller må noe slikt implementeres i programvaren

I vår modell lagres instruktøren for en gruppeaktivitet som fremmednøkkelen *instrukt_ID* i *Gruppeaktivitet*-tabellen. En `UNIQUE`-restriksjon kan kun forhindre identiske verdier, ikke overlappende *intervaller*, og kan dermed ikke fange opp to gruppeaktiviteter med samme instruktør i overlappende tidsrom.

Løsning via triggere. Vi har implementert to triggere som håndhever regelen direkte i databasen:

- `check_instruktør_overlapp_insert` – kjøres `BEFORE INSERT` på *Gruppeaktivitet*.
- `check_instruktør_overlapp_update` – kjøres `BEFORE UPDATE` på *Gruppeaktivitet*.

Triggerene sjekker om det finnes en annen rad i *Gruppeaktivitet* med samme *instrukt_ID* og *dato* der *start/slutt*-intervallene overlapper, og avbryter med `RAISE(ABORT, ...)` i så fall.

Konklusjon. Regelen kan ikke uttrykkes med den relasjonelle modellen alene, men er fullt ut håndterbar i databasen med to triggere på *Gruppeaktivitet*.

4.2 Det samme spørsmålet for en bruker. Kan dette løses i modellen eller må det gjøres i programvaren?

Idrettslagstimer er forhåndsbooket for hele idrettslaget og krever ingen individuell påmelding – *møter_til_idrett* er kun oppmøteregistrering. Overlappsregelen er dermed utelukkende relevant for individuelle påmeldinger til gruppeaktiviteter via *påmeldt_til*.

To gruppeaktiviteter for samme profil kan kollidere dersom begge har samme *dato* og overlappende *start/slutt*. Dette kan ikke uttrykkes med `UNIQUE` eller`CHECK`, men *kan* implementeres med triggere.

Løsning via triggere. Vi har implementert to triggere som håndhever regelen direkte i databasen:

- `check_bruker_overlapp_påmeldt_insert` – kjøres `BEFORE INSERT` på *påmeldt_til*.
- `check_bruker_overlapp_påmeldt_update` – kjøres `BEFORE UPDATE` på *påmeldt_til*.

Triggerene gjør en `JOIN` fra *påmeldt_til* til *Gruppeaktivitet* for å hente *dato/start/slutt*, og avbryter med `RAISE(ABORT, ...)` dersom profilen allerede er påmeldt en annen gruppeaktivitet i overlappende tidsrom.

Konklusjon. Regelen kan ikke uttrykkes med den relasjonelle modellen alene, men er fullt ut håndterbar i databasen med to triggere på *påmeldt_til*. Vi har valgt å implementere den direkte i SQL-scriptet, slik at applikasjonslaget ikke trenger å håndheve den manuelt.

4.3 Fra hvilket usecase testes det for / opprettes utestengelse (svartelisting)? Det trenger ikke være en av de oppgitte usecasene.

Utestengelse involverer to steg: (1) *oppretting* av prikker som til slutt utløser utestengelsen, og (2) *sjekk* av om en profil er utestengt.

Oppretting. Prikker gis til profiler som er påmeldt en gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start. Grunnlaget for dette er **brukstilfelle 3** (registrering av oppmøte): det er oppmøteregistreringene i *møter_til_gruppe* som gjør det mulig å avgjøre hvem som *ikke* har møtt. Selve tildelingen av prikker skjer i et eget brukstilfelle; *tildeling av prikker etter oppmøtefrist*, som etter at oppmøtefristen har passert (5 minutter før start), sammenligner *påmeldt_til* med *møter_til_gruppe* og oppretter en rad i *Prikk* for de som ikke har møtt. Når en profil har fått ≥ 3 prikker innenfor de siste 30 dagene, er den utestengt.

Sjekk. Utestengelsessjekken er implementert direkte i databasen ved hjelp av to triggere på tabellen *påmeldt_til*:

- `check_utestengelse_insert` – kjøres `BEFORE INSERT`
- `check_utestengelse_update` – kjøres `BEFORE UPDATE`

Dersom profilen har ≥ 3 prikker de siste 30 dagene, avbrytes operasjonen med `RAISE(ABORT, ...)`. Sjekken trenger derfor ikke implementeres i applikasjonslaget – databasen håndhever regelen automatisk ved hvert forsøk på påmelding.

Brukstilfelle 6 demonstrerer svartelistingsmekanismen eksplisitt, men den naturlige triggeren i et produksjonssystem er brukstilfellet for prikk-tildeling beskrevet over, kombinert med triggerne i databasen som blokkerer påmelding ved utestengelse.

4.4 Statistikk er det noe som må lagres eksplisitt eller går det an å bare gjøre queries for å få svar? Diskuter dette.

Vi mener det meste av statistikken som er relevant for SiT kan utledes med vanlige SQL-spørringer mot databasen. Dette vil også gjøre det enklere å oppdatere statistikken hvis det skjer endringer i databasen.

Eksempler på statistikk som kan beregnes med queries.

- **Antall påmeldte per økt:** `COUNT(*)` på *påmeldt_til* gruppert på gruppeaktivitet.
- **Hvem som deltok på en gitt trening:** spørring mot *møter_til_gruppe*.
- **Personen(e) med flest gruppetimer i en måned** (brukstilfelle 7): `COUNT(*)` på *møter_til_gruppe* gruppert på *profil_ID*, filtrert på måned.
- **Par som trener sammen** (brukstilfelle 8): selvjoin på *møter_til_gruppe*.
- **Ukeplan** (brukstilfelle 4): spørring mot *Gruppeaktivitet* filtrert på datoperiode.
- **Prikkstatus og utestengelse:** beregnes fra *Prikk*-tabellen ved å telle prikker innenfor de siste 30 dagene.

Når kan eksplisitt lagring likevel lønne seg? I større systemer kan det være fordelaktig å lagre statistikk eksplisitt. Med store datamengder kan aggregerings-spørringer bli trege; forhånds-beregnet statistikk, for eksempel i form av materialiserte views eller oppsummeringstabeller, gir da raskere tilgang. Dersom gamle data slettes eller endres over tid, kan historisk statistikk gå tapt med mindre den er lagret eksplisitt. I tillegg kan periodiske rapporter beregnes på forhånd og lagres, slik at de er raskt tilgjengelige uten å kjøre tunge spørringer på nytt.

Konklusjon. Statistikk trenger *ikke* lagres eksplisitt i vår modell. For SiTs skala er datamengdene moderate, og alt kan utledes av de normaliserte tabellene med vanlige SQL-spørringer. I et større system ville det likevel vært aktuelt å vurdere eksplisitt lagring av statistikk for ytelse og bevaring av historiske data.